

C++ Standard Library Issues Resolved Directly In Albuquerque

Doc. no. P0864R0

Date: Revised 2017-11-11 at 01:05:20 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Immediate Issues

[2958^{\(i\)}](#). Moves improperly defined as deleted

Section: 23.4.2 [[pairs.pair](#)], 23.5.3.2 [[tuple.assign](#)] **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2017-05-05 **Last modified:** 2017-11-10

Priority: 2

View other [active issues](#) in [[pairs.pair](#)].

View all other [issues](#) in [[pairs.pair](#)].

Discussion:

LWG [2729](#) constrained many `pair` & `tuple` assignment operators to "not participate in overload resolution" and "be defined as deleted if." As discussed when LWG reviewed [2756](#), it is undesirable to require that a move constructor or assignment operator be "defined as deleted," since it is unclear whether that operation should be *implicitly* deleted, and therefore not participate in overload resolution, or *explicitly* deleted and therefore impede overload resolution for usages that would otherwise find copies.

[2017-07 Toronto Wed Issue Prioritization]

Priority 2

[2017-11 Albuquerque Wednesday issue processing]

Move to Immediate

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 23.4.2 [[pairs.pair](#)] as indicated:

```
pair& operator=(pair&& p) noexcept(see below);
```

-21- *Effects:* Assigns to first with `std::forward<first_type>(p.first)` and to second with `std::forward<second_type>(p.second)`.

-22- *Remarks:* This operator shall ~~be defined as deleted~~ not participate in overload resolution unless `is_move_assignable_v<first_type>` is true and `is_move_assignable_v<second_type>` is true.

[...]

2. Edit 23.5.3.2 [\[tuple.assign\]](#) as indicated:

```
tuple& operator=(tuple&& u) noexcept(see below);
```

-5- *Effects:* For all i , assigns `std::forward<T $i$ i to get<math>i.`

-6- *Remarks:* This operator shall ~~be defined as deleted~~ not participate in overload resolution unless `is_move_assignable_v<T $i$  is true for all  $i$ .`

[...]